Check for updates

Research Article

Neuro-Symbolic Enterprise Optimization

The European Journal on Artificial Intelligence
1–16
© The Author(s) 2026

Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/30504554261422145
journals.sagepub.com/home/eai

Sytze P. E. Andringa[1] and **Neil Yorke-Smith**[1]

Sage | IOS Press

Abstract

Simulation–optimization is often used in enterprise decision-making processes, both operational and tactical. This paper proposes a data-driven, declarative approach to enterprise optimization. In our generic simulation–optimization approach, a declarative constraint satisfaction problem (CSP) is automatically customized given a simulation model and a problem instance. We construct the declarative model by training a neural network on the simulation model and embedding the trained network in the CSP. The approach allows the embedding of a wide variety of complex problems, is able to handle multi-objective problems, is flexible to changing multiple objectives simultaneously, and allows the modeler to focus on *what* problem needs to be solved by a computer instead of *how* the computer should solve it. We demonstrate the value of this neuro-symbolic approach in experiments on two problem domains. Furthermore, we show how low discrepancy sampling, weighted composite objectives, and linear activation functions can improve the performance of neural embeddings in CSP.

Keywords

enterprise simulation, constraint programming, deep learning, simulation–optimization

Received: August 11, 2023; accepted: January 23, 2026

1 Introduction

Managerial decisions are often an attempt at balancing multiple objectives, when critical decision factors are unclear and causal effects are hard to explain (Tan et al., 2020). Hence, decision makers (DMs) tend to use various tools, such as simulation, to gain insight into how an enterprise might behave in complex or future scenarios (Kampik and Najjar, 2019; Vernadat, 2020). With simulation, various managerial interventions can be analyzed for the likely outcomes. DMs that use simulation models in practice are often interested in finding "optimal" inputs with respect to an observed "problem"—an undesired property of the system that can be tackled by taking intervening action (Bock et al., 2018; Laguna and Marklund, 2013).

Finding these inputs by exhaustively trying out them all through simulation is feasible with only limited success on complex simulation processes, for the number of possible inputs grows exponentially with respect to the number of input parameters (Laguna and Marklund, 2013). On the other hand, using a pure optimization model to compute the "optimal" inputs—that is, omitting simulation—can be incapable of capturing all complexities and dynamics of a system (Laguna and Marklund, 2013). Hence *simulation–optimization* (SO) seeks to combine the benefits of both by using simulation to represent the actual system and optimization to find optimal simulation inputs (Teerasoponpong and Sopadang, 2021), for example, to determine strategic marketing decisions (Schlosser et al., 2019).

This article situates *enterprise optimization* (Grossmann, 2014) as optimization—that is, forming strategies to obtain some outcome—put in the context of managing an enterprise. A wide variety of enterprise optimization strategies and tools exist, for which an overview can be found in Varma et al. (2007).

[1]STAR Lab, Delft University of Technology, Delft, The Netherlands

Corresponding Author:
Neil Yorke-Smith, STAR Lab, Delft University of Technology, TU Delft, P.O. Box 5031, 2600 GA, Delft, The Netherlands.
Email: n.yorke-smith@tudelft.nl

A mapping from problem description to a formal SO model is non-trivial. From a modeler's perspective, existing enterprise modeling (EM) methods offer only tenuous concepts of "problems" (Bock et al., 2018). From a computational perspective, optimization models can be difficult for a DM with a non-technical background to understand when the model is heavy with formal mathematics.

A path to model problems in a more understandable but computer-parseable format is with constraint programming (CP; Andringa and Yorke-Smith, 2021). CP is a declarative paradigm for defining combinatorial optimization; it allows one to describe diverse real-world problems through constraints, that is, statements which pose some relation among the problem's variables (Rossi, 2000). Notably, the expressiveness of CP means its formal models can be more human-like than, for instance, mixed integer programming optimization models.

This article addresses the modeling challenge of SO for the enterprise by means of CP and automated modeling. The simulation model is represented by a "model of a model," that is, *meta-model* (Jin et al., 2001). Meta-modeling techniques range from descriptive representations of ontological concepts to algorithms to make faster approximations of complex computer code (Atkinson and Kuhne, 2003; Jin et al., 2001). We focus on the latter by representing the simulation model with a neural network (NN). NNs are capable of learning behavior of complex systems and require little engineering by hand, making them applicable in many domains (Bartolini et al., 2011; LeCun et al., 2015; Teerasoponpong and Sopadang, 2021). Our approach is to train an NN on simulation data and *automatically embed* the NN into the CP model; we then solve multiple sampled instances of the CP model to obtain an approximation of the corresponding pareto frontier of the underlying enterprise optimization problem.

We argue that embedding an NN into a declarative model adds additional flexibility during the problem-solving process, since a change of objective can be evaluated without making additional simulation calls. This flexibility can be desirable in various situations, for example, when:

(1) The DM does not know all specifications of the problem it wants to solve (e.g., information by a third party is required), but does know how the system currently behaves, and would like to make preparations such that problems can be solved on-the-go without making computationally expensive simulation calls.
(2) The DM has a large collection of problems that needs to be solved, making setting up a separate feedback loop for every problem infeasible.
(3) The DM is not able to do simulation during the problem-solving phase.

We argue that the proposed methodology can be applied to a wide range of problems due to its generality. The two main components of the approach—CP and machine learning—are both effective in modeling complexity. CP is able to model complex problem structures, whereas (deep) machine learning is able to capture complex relationships between variables.

A preliminary report of this work appeared at a conference (Andringa and Yorke-Smith, 2021). That initial paper showed how easy-to-understand CP programs can be constructed from descriptive problem descriptions and how these can be evaluated to make strategic decisions. In this full article, we further detail the approach and particularly improve performance and show generality:

(1) The use of low discrepancy sampling to determine what weights to set to objectives, to obtain higher quality problem instances.
(2) The use of various output activation functions in the trained network, for more efficient constraint satisfaction problem (CSP) embeddings of the network.
(3) The direct optimization toward a composite weighted objective instead of considering thresholds.
(4) More detailed computational experiments and experiments on a new problem domain.

The remainder of the article is structured as follows. Section 2 discusses relevant literature, approaches, and techniques. Section 3 discusses our approach in more detail. Section 4 shows the applicability and performance of the approach through a set of experiments and discusses the results. Section 5 concludes with a discussion of future work.

2 Background and Related Work

2.1 Constraint Programming (CP)

CP is an expressive yet practical approach to optimization, used in a wide variety of applications (Rossi, 2000; Wallace, 2020). It solves constraint satisfaction problems, which consist of a set of variables, each with a domain of permitted values, and a set of constraints specified in a logical formalism. A solution to a CSP is an assignment of a value to every

Table 1. Mathematical Programming Versus Constraint Programming (IBM, 2021).

Mathematical Programming	Constraint Programming
Typically restricted to linear and quadratic problems	Typically discrete but also continuous problems
Proves optimality with techniques such as a lower-bound proof provided by cuts and linear relaxation	Proves optimality by showing that no better solution than the current one can be found
Algebra as theoretical basis	Logic as theoretical basis
Requires that the model falls in a well-defined mathematical category	Does not make assumptions on the mathematical properties of the solution space
Is specific to a class of problems whose formulation satisfies certain mathematical properties	Has no limitation on the arithmetic constraints that can be set on decision variables

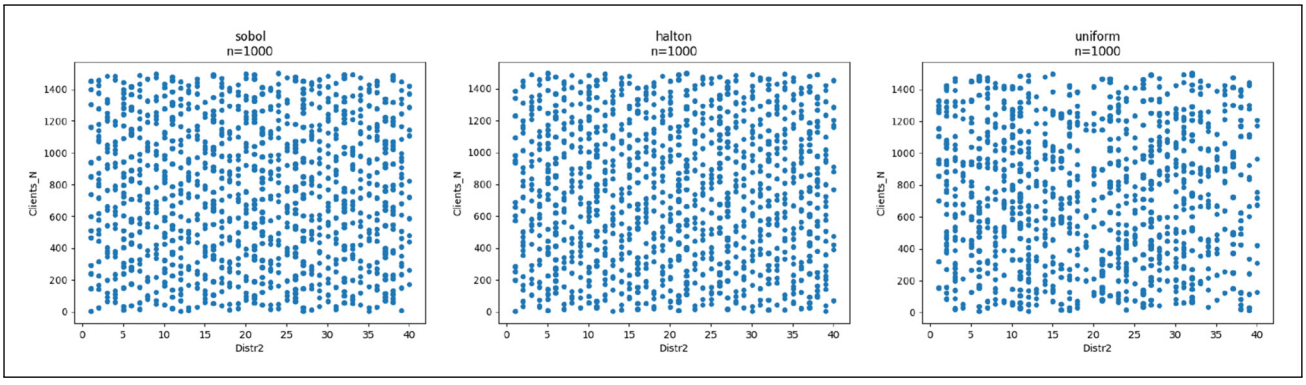


Figure 1. Demonstration of two low-discrepancy sampling methods (Sobol and Halton) as opposed to uniform sampling for 1,000 data points. The plot visualizes how both low-discrepancy sequences have less gaps and clusters.

variable from its domain, such that all the constraints are satisfied. *Soft CSPs* permit the constraints to be satisfied to a degree, rather than binary satisfaction. Constraint optimization problems (COPs) include an objective function. For both CSP and COPs, the resulting model is passed to a (black-box) solving algorithm. A wide variety of such solvers, complete and incomplete, are available, such as *or-tools* from Google (Perron and Furnon, 0000).

A common optimization methodology is mathematical programming (MP), archetypally mixed integer linear programming. Although CP and MP share a similar model-and-solve paradigm—there is a set of decision variables, a set of constraints, and often an objective function to maximize or minimize—CP is a more expressive formalism. It can be thought of as a generalization of mixed integer programming to non-linear and non-arithmetic constraints (Wallace, 2020). Table 1 provides an overview of the differences between CP and MP. Notably for the purpose of this article, the expressive nature of CP allows for models which are closer to human-level expression of problems (Buscemi and Montanari, 2008).

2.2 Quasi-Monte Carlo Sampling

In our preliminary work, we set weights for the objectives in the composite objective (see Section 3 for more details) through uniform random sampling. As an alternative approach, we consider Quasi-Monte Carlo sampling. Quasi-Monte Carlo is attained by using quasi-random—also called low-discrepancy—sequences in Monte Carlo sampling (Caflisch, 1998). Low discrepancy sequences aim to sample a number of points that efficiently uniformly cover some d-dimensional space (Cervellera et al., 2014). This concept is visualized in Figure 1. While quasi-random sequences improve the convergence of numerous applications, it is by no means trivial to enhance the convergence of all Monte Carlo sequences. In this article, we put focus on two low-discrepancy sequences, namely the Halton (1960) and Sobol (1967) sequences. For more details on Monte Carlo and Quasi-Monte Carlo sampling, we refer to Lemieux (2009).

2.3 SO & Empirical Decision Model Learning

As discussed in the introduction, the technique described in this article falls under the category of simulation–optimization. More specifically, a function–estimation-based approach, where an NN is fitted on the underlying system.

The literature on SO tends to favor evolutionary search to find optimal simulation inputs (Laguna and Marklund, 2013; Teerasoponpong and Sopadang, 2021; Yalcin et al., 2022). This black-box SO approach can be convenient when

Table 2. Black Box Optimization Versus Empirical Model Learning (Lombardi et al., 2017).

Black Box Optimization	Empirical Model Learning
Designed for problems without a complex combinatorial structure (discrete variables and non-trivial constraints)	Tends to provide the best results for problems with a complex combinatorial structure
Relies on performing simulation during the search process	Simulation time has no direct impact on the solver performance
Function that describes the system is a black box	No black box assumption, allowing exploitation of its structure during the search process

a quick evaluation of the simulation model is possible. However, if simulation is expensive—frequently the case with simulators—advanced techniques are necessary to limit the number of simulation calls (Lombardi et al., 2017).

Instead, we adopt an empirical decision model learning (EML; Lombardi et al., 2017), which differs in various ways from black-box optimization as shown in Table 2. EML is a methodology that learns relations between decidables (controllable variables) and observables (uncontrollable parameters) from data, and encapsulates these relations into components of an optimization model. Data can be harvested from a real system or a predictive model, such as a simulator. The encapsulation can be done by machine learning methods, for example, an NN.

2.4 Closely Related Approaches

Combining CP with EM is recognized as valuable in the literature. Several studies couple CP with the closely-to-EM-related Business Process Management to solve (complex) scheduling and planning problems, as the declarative paradigm allows them to conveniently embed their custom constraints (Jimenez-Ramirez et al., 2012; Wiśniewski et al., 2021). However, these studies typically require the DM to describe the flow of the system in the CP model. In contrast, this article considers a more general approach and studies CP models that merely describe the problem to solve. The DM is therefore not required to describe how the system operates. Differently stated, the DM can treat the simulation model as a black box and can focus on describing what relation between simulation inputs and outputs should be satisfied. It does not have to consider details of internal processes, as these are modeled by means of a simulation model. We believe these black-box properties make the approach applicable to a wide variety of problems, since the only requirement is access to a simulation model and a problem that can be defined with CP, given the simulation inputs and outputs.

3 Methodology

Figure 2 provides an overview of our approach. It has two main characteristics. First, we use simple—understandable for those with a non-technical background—CP to model the problem. Second, we represent the simulation model automatically in an optimization model by representing it as an NN. This section details the two main components of our approach. Then, multiple objectives and the solving procedure are discussed.

In this article, the simulation model is assumed to represent a complex real-world system. It can, for example, be derived from an enterprise model (Vernadat, 2020). How to design a qualitative simulation model is not in the scope of this article: the reader is referred to Kampik and Najjar (2019) and Laguna and Marklund (Laguna and Marklund, 2013).

3.1 The NN

The first main component is a meta-model, derived from a simulation model representing the behavior of a system. This allows the DM to consult the meta-model instead of the simulation, such that simulation calls do not have to be made during the solving procedure. Deriving such a meta-model poses a tradeoff. On the one hand, it should reflect the simulation model properly. On the other hand, it should be convenient enough to allow optimization. This dilemma corresponds with a fundamental computational reality, namely the tradeoff between expressiveness and tractability. A model should be detailed enough such that it makes sense—it is expressive—but not too detailed because otherwise computations cannot be made feasibly—it lacks tractability (Brachman and Levesque, 2004). This article proposes using NNs as meta-models for several reasons:

- NNs are able to learn behavior of opaque or very complex systems, without requiring detailed knowledge of their components and interactions (Bartolini et al., 2011; LeCun et al., 2015). They are capable of dealing with both the non-linearity and uncertainty of the underlying system (Teerasoponpong and Sopadang, 2021).
- NN embeddings in optimization models have shown good performance in comparison to other combinations of optimization methods with machine learning techniques (Lombardi et al., 2017).

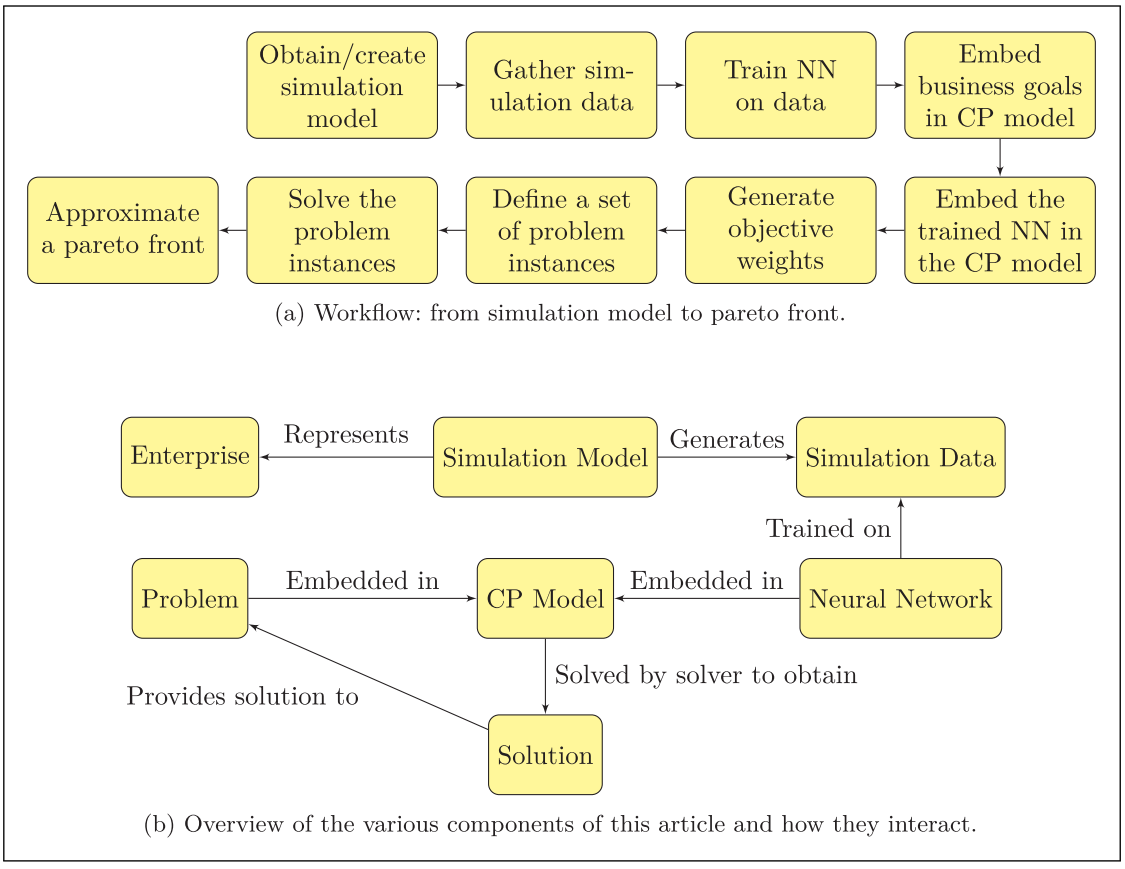


Figure 2. Approach summarized. A simple NN is trained on simulation data. Next, business goals and trained NNs are embedded in the CP model. By utilizing soft constraints and various weight and threshold parameters, an approximation of the Pareto front for the various objectives is formed. (a) Workflow: from simulation model to Pareto front. (b) Overview of the various components of this article and how they interact. *Note.* NN = neural network; CP = constraint programming.

- Compared to other machine learning techniques, NNs are very capable in identifying what features are interesting and excel at handling high-dimensional input data. The result is that NNs require very little engineering by hand and are applicable in many domains (LeCun et al., 2015).

3.1.1 Activation Function. The activation functions also play a crucial role in NN (Dubey et al., 2021). A wide variety of activation functions exist, such as polynomial, adaptive, and rectified linear unit-based functions. For a comprehensive survey and benchmark study on activation functions, we refer to Dubey et al. (2021). The logistic sigmoid activation function is a popular and traditional non-linear activation function:

$$f(x) = \frac{1}{1 + e^{-x}}. \tag{1}$$

Using a convex activation function allows solvers to solve by means of convex optimization, which tends to converge to a solution much faster than non-convex optimization (Nesterov and Nemirovskii, 1994). Although sigmoid tends to perform very well for many deep learning applications, it includes an exponential quadratic term, which has a major impact on the solving time as it introduces non-linearity. Alternatively, one could consider activation functions that are linear—while retaining properties that are often found in non-linear activation functions—and might thus be more fitting for a CP embedding.

For this reason, in this article, we study the performance of two alternative approaches to the sigmoid activation function. The first is a quadratic variant *power2*, which replaces the e^{-x} term by x^2:

$$f(x) = \frac{1}{1 + x^2}. \tag{2}$$

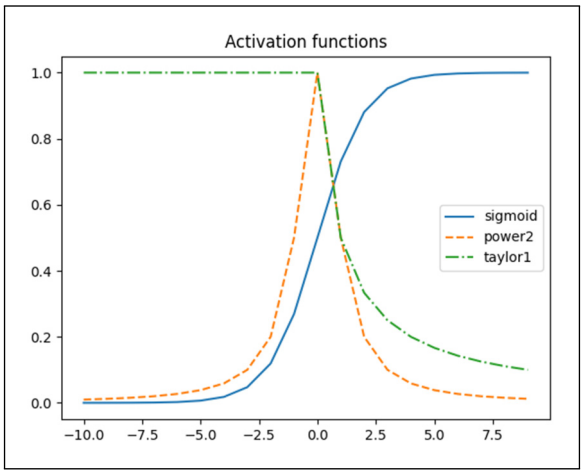


Figure 3. The shape of three activation functions.

The second is a semi-linear variant *taylor1*, based on the first term of the Taylor approximation of the sigmoid function:

$$f(x) = \frac{1}{1 + \max(0, x)}. \tag{3}$$

Figure 3 visualizes the shape of the discussed functions. It shows all three are non-linear, which is an often desired property of activation functions (Dubey et al., 2021). It is noteworthy that the sigmoid is the only injective function: no output can be achieved in multiple ways. Our experiments will examine the tradeoff between the linearity of power2 and taylor1 versus the injective sigmoid.

3.2 The Constraint Program

Besides the NN, the second main component we need is the CP model. It intends to represent a problem to be solved. There are three main aspects to be identified in problems, namely (1) an as-is enterprise optimization scenario which is considered to be non-optimal, (2) uncertainty about what decision would lead to the preferred situation, and(3) a preferred situation to achieve (Bock et al., 2018). We construct the CP model accordingly:

(1) The current situation is *modeled* by a simulation model and *represented* by a trained NN. An NN can be embedded into a CP model, which allows expressing constraints over the NN output (Bartolini et al., 2011).
(2) Uncertainty is also incorporated by the NN. Training an NN creates a predictive model. In other words, the NN is used to tackle the uncertainty involved in the problem by giving insight into the correlation between variables and parameters in the decision model.
(3) The preferred situation is incorporated by the objective function. In our approach, the objective function is expressed by composite weighted objectives.

It is reasonable to believe most—if not all—problems can be described by elements also found in CP. This is based on the theoretical conceptualization of a problem by a meta-model presented in Bock et al. (2018), where the concept of a problem is decomposed. The interconnection between this decomposition and CP is shown in Table 3.

As noted in Section 2, constraints expressed in natural language can be formalized in CP at a higher level compared to other optimization methods. The result is that constraints expressed by the DM describing what solution it aims to seek can be conveniently expressed in the CP model. For example, CP makes it convenient to state some relationship between variables that should always be satisfied, such as “machine A should never produce more than machine B” or “department C should always have more employees than department D”; so-called *global constraints* are particularly useful (Wallace, 2020). Figure 1 shows an example CP model that highlights its readability. We use the modeling language MiniZinc; MiniZinc conveniently connects to various solvers (Nethercote et al., 2007).

Our approach embeds an NN into a CP model by expressing the value of a node as a function of the values of nodes in a previous layer. This is based on the concept of *neuron constraints*, which allow one to encode complex networks

Table 3. Concepts From the Meta-Model Presented by Bock et al. (2018) and Their CP Equivalents.

Problem Conceptualization Meta-Model	CP Equivalent
Factual aspect	Constraint
Goal	Objective function
Value	Variable
Stakeholder preference	Soft-constraint
Possible action	Decision variable

Note. A factual aspect describes something regarded as true, that is, a constraint. A goal is a metric to improve upon that is expressed by other metrics, that is, an objective function. Multiple stakeholders can be modeled by soft constraints, which provides a foundation to model individual preferences (Schiex, 1992). An action describes something that can be undertaken, hence corresponds with a variable that can be decided upon. CP = constraint programming.

Algorithm 1. Pseudocode for the restaurant CP model. It closely matches a MiniZinc implementation.

```
include NN
float[] buying_constraints = [c1,...,cn]
var[] buyingStrategy = [b1,...,bn]
var spoilage
var success
constraint forall (i in 1..n) (0 <= bi <= ci)
solve maximize (success * success_weight - spoilage * spoilage_weight)

```

using a limited number of basic components (Bartolini et al., 2011). An example of such an embedding can be found in Appendix B.

3.3 Multi-Objective Approach

So far, we have a CP declarative model with an embedded NN. We next discuss how to obtain solutions, and moreover, what kind of solutions are to be sought.

Most methods on multi-objective decision making in business analytics are *posteriori*, in the sense that they obtain preference information—how much each objective is preferred over the others—from the DM *after* computing solutions (Yalcin et al., 2022). These methods are useful when the DM is interested in the scope of actions she can take—particularly useful when there is no single dominating solution—as multiple solutions are provided instead of a single one. In CP, preference information *is* the input, since the CP model is asked what inputs are necessary to satisfy certain objectives. As such, the approach can be made posteriori by evaluating a set of problems covering the variety of possible preferences.

We put our focus on approaches that do not require preference information beforehand. In our preliminary report (Andringa and Yorke-Smith, 2021), this was realized by a *possibilistic CSP*, where each objective had a corresponding weighted soft constraint (WSC; Schiex, 1992). A WSC has a weight, indicating the importance of its being satisfied, and a threshold. For a maximization objective, the WSC is satisfied if the objective value exceeds the threshold, and for a minimization objective if it does not. Then, random weights and thresholds are generated in order to create a set of problem instances. Next, this set of problems is solved. Their dominating solutions form the output.

For the experiments in this article, we considered to optimize toward weighted composite objectives, that is, we do not specify thresholds as the objective function equals a scaled sum of the objectives. The main motivation for a preference over this "direct" approach of the possibilistic CSP is, as the name already states, a more direct representation of the goal in this experiment, namely converging toward a Pareto front. This makes it easier to understand and construct the CP models, as it does not require a threshold parameter.

For problems that need to preserve the binary pass/fail properties of the problem, a possibilistic CSP approach might be more fitting. A trivial example is when the DM is interested in optimizing some target value y to exceed threshold t, but does not care by how much y has exceeded t. This makes it a modeling design choice as it relates to *what* we want to solve and not *how* we aim to solve it. As such, a comparison between the two by experiments makes little sense as we are solving different problems, and thus is not included in this article.[1]

Furthermore, instead of generating the weights for individual objectives in the composite objective uniformly at random, we consider generating these by deterministic low-discrepancy sequences. The idea is that this improves the quality of the

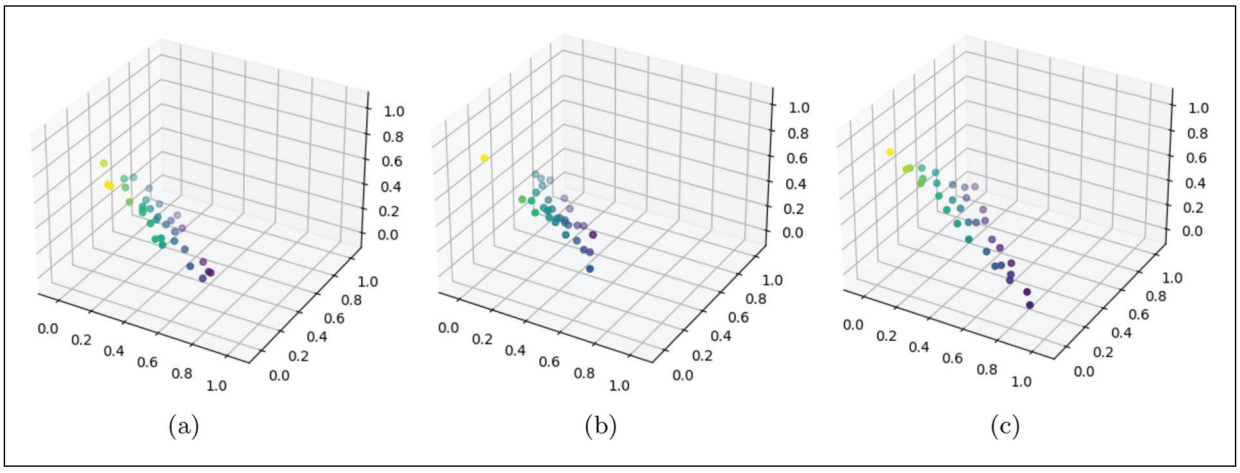


Figure 4. 3D plots to show the effects of Montgomery sampling compared to uniformly sampling and a normalized Halton sampling. 32 three-dimensional data points were generated. The two normalized variants were implemented by first generating the points according to their underlying sequence, and then for each point, dividing each of its value by the sum of its values such that the summation constraint was satisfied. Montgomery sampling tends to outperform the other two in terms of L2* discrepancy (Zhou et al., 2013). Visually, this can also be observed, as the triangle-shaped solution space is more easily noticeable for Montgomery sampling than for the other two methods. (a) Normalized Halton L2* discrepancy = 0.135. (b) Normalized Uniform L2* discrepancy = 0.155. (c) Montgomery L2* discrepancy = 0.119.

Algorithm 2. Montgomery sampling.

$n \leftarrow$ number of data points to sample
$d \leftarrow$ number of dimensions to sample
$H^{sub} \leftarrow H^{d-1}_{(d-1)!\cdot n}$ ▷ Get the first $(d-1)! \cdot n$ points of the $d-1$ dimensional Halton Sequence
$H^m \leftarrow \emptyset$
$i \leftarrow 0$
while $|H^m| < n$ **do** ▷ While not enough points are generated
  $x \leftarrow H^{sub}(i)$ ▷ Sample a new point of dimension $d-1$
  $s \leftarrow 1 - \sum_{d_i \in d-1} x_{d_i}$ ▷ Store 1 minus the sum of the $d-1$ points
  if $s \geq 0$ **then** ▷ Discard the point if the sum exceeds 1
    $x_d \leftarrow s$ ▷ Set d-th coordinate of x to the sum of the $d-1$ points
    $H^m \leftarrow H^m \cup x$
  end if
  $i \leftarrow i + 1$
end while
return H^m

Pareto front, as the generated CSP instances are more evenly spread in the objective space. Since the sum of the sampled weights should sum to one, this article considers Montgomery sampling (see Algorithm 2), a variant on low-discrepancy sampling. Montgomery sampling takes a $d-1$ dimensional low discrepancy sequence and maps it to a d-dimensional low discrepancy sequence, while preserving this summation constraint. A demonstration of its performance can be found in Figure 4.

It should be noted that Sobol and Halting sampling sequences tend to become infeasible to compute for higher dimensions (around approximately $d > 10$). This can be problematic for problems with many objectives. There are several ways to overcome this issue. Suppose p is the threshold from which Montgomery sampling becomes infeasible, and we would like to sample $d > p$ dimensions. One way would be to sample the first q dimensions by means of Montgomery sampling and use uniform sampling to sample the remaining $d - p$ samples. Alternatively, we could also consider modifying the algorithm to sample the $d-1$ samples in sets $q_i \in Q$, such that $q_i \leq p, \forall q_i \in Q$ and $d - 1 = \sum_{q_i \in Q} q_i$. The last dimension can be added similarly to Montgomery sampling, by setting it equal to 1 minus the sum of the sampled digits. Evaluating these approaches experimentally is left as future work.

Table 4. Performance Indicators.

Metric	Assessment	Description
MSE	NN accuracy	Mean squared difference between approximated output and real output on a test set
Runtime	Performance	Average time for solving one CSP instance
Front occupation	Solution quality	Amount of Pareto optimal solutions found
Front spread	Solution diversity	Maximum distance between any two points in the set of solutions

Note. More details on the multi-objective metrics can be found in Audet et al. (2021). MSE = mean squared error; NN = neural network; CSP = constraint satisfaction problem.

4 Experimental Validation

This section assesses the approach described in Section 3 by means of three experiments. Experiments 1 and 2 are based on a restaurant simulation. These show the effect of various design choices. Experiment 3 is based on the Food Wars simulation (Rasmussen and Wilensky, 2019), where we take a governing role over foragers with an aim to make them less hostile. The approach is assessed by various performance metrics, which are summarized in Table 4. These assess the runtime, accuracy, solution quality, and solution diversity.

Two further experiments, one based on a simulation model derived from a DEMO model and one based on a simulation study performed by other work, are reported in Andringa (2021). These two experiments highlight the generality of the approach by taking a problem description or enterprise model as input. Source code is available under MIT licence at https://www.doi.org/doi/10.4121/26776b70-225b-4091-ace3-e57324d0eaf1.

The Food Wars domain is more relevant to the focus of this article than the supply chain experiment from our preliminary work (Andringa and Yorke-Smith, 2021). Our previous work demonstrated the applicability and (initial) generalizability of the approach. Therefore, the supply chain experiment fitted well as its application is closely related to enterprise problems. However, this article puts more emphasis on performance and the utility of the approach. The supply chain experiment is less suitable for performance analysis as its underlying simulation environment is computationally exhaustive. Additionally, its output has large variance, since each initialization shuffles all connections in the supply chain network. This made generating sufficient simulation data an exhaustive approach. In contrast, the Food Wars domain allows for much better performance examination. Its corresponding simulator is much less computationally expensive, and its outputs have less variance. Furthermore, it is reasonable to classify the domain as being "complex," due to its agent-based nature.

4.1 Restaurant—Problem Description

As stated, Experiments 1 and 2 use a restaurant simulation, introduced in Andringa and Yorke-Smith (2021). The restaurant buys ingredients periodically according to some buying strategy and processes them into various dishes. Its buying strategy is represented by an integer per resource that indicates the quantity being bought each period. Buying strategies have limitations, for example, due to seasonal ingredients. Resources can spoil if stored for too long. The restaurant has two objectives that characterize a tradeoff: it should not buy too many resources in order to minimize spoilage, but also should not buy too little in order to maximize the number of successful orders. The restaurant is interested in how its buying strategy affects its spoilage and success ratio.

The original problem has two objectives; extra objectives can be added by considering a spoil ratio per ingredient, and a success rate per dish.

4.2 Experiment 1: CSP Weight Sampling

The purpose of this experiment is to assess the performance of using low-discrepancy samplers to determine the weights for constructing the composite objective. As discussed in Section 3.3, our preliminary approach generated the weights uniformly at random. In this article, we adopt the use of Montgomery sampling to determine these. As shown in Table 5, low discrepancy sampling outperforms uniform sampling both in solution diversity (indicated by the higher front spread) and solution quality (indicated by the higher front occupation).

As opposed to uniform sampling, Montgomery sampling is deterministic. The result is a more stable method to generate problems. This can be seen as an additional advantage of Montgomery sampling over the random approaches.

Table 5. Experiment 1: Performance Indicators of Various Weight Sampling Methods.

Sampler	Objectives	Repetitions	FO	FS	Runtime	Repetitions
Uniform	2	10	4.6	0.19	1.2 ± 0.51	10
Montgomery–Sobol	2	5	6	0.25	1.1 ± 0.52	5
Montgomery–Halton	2	5	5	0.25	1.1 ± 0.54	5
Uniform	3	13	4.5	0.75	1.3 ± 0.81	13
Montgomery–Sobol	3	4	7	0.77	1.2 ± 0.81	4
Montgomery–Halton	3	4	6	0.75	1.4 ± 0.98	4
Uniform	4	13	5.9	0.47	2.5 ± 2.7	13
Montgomery–Sobol	4	4	9	0.86	3.3 ± 4.3	4
Montgomery–Halton	4	4	9.8	0.86	3.8 ± 4.1	4
Uniform	5	12	13	0.56	2.4 ± 1.4	12
Montgomery–Sobol	5	3	17	1.3	2.1 ± 1.4	3
Montgomery–Halton	5	3	17	1.27	2.5 ± 1.8	3
Uniform	6	10	6.7	1.09	3.2 ± 2.2	10
Montgomery–Sobol	6	5	7.8	1.3	2.6 ± 2.0	5
Montgomery–Halton	6	5	7	1.2	2.7 ± 2.1	5

Note. The best performing method is shown in bold. The experiment was repeated additionally for the uniform sampling to compensate for the random nature.FO = front occupation; FS = front spread.

Table 6. Experiment 2: Performance of Three Activation Functions.

Repetitions	Solver	Architecture	Hidden Layers	Layer Width	Runtime	MSE	FO	FS
25	Gurobi	Pow2	2	5	0.55	0.0033	4.2	0.12
6	Gurobi	Taylor1	3	7	1.0	0.0014	5	0.18
11	Gurobi	Taylor1	2	10	1.3	0.00080	5.2	0.25
14	Gurobi	**Pow2**	2	7	**1.6**	0.0016	**5.8**	**0.26**
9	Gurobi	**Taylor1**	2	12	**4.5**	**0.00071**	5.6	0.22
13	Gurobi	Pow2	2	10	6.8	0.00071	7.8	0.27
12	JaCoP	Sigmoid	2	5	53	0.0026	5	0.077
8	JaCoP	Pow2	2	5	99	0.0031	4.1	0.09
13	JaCoP	Sigmoid	2	7	116	0.0012	7.5	0.16
8	JaCoP	Pow2	2	7	183	0.002	4.8	0.17

Note. Here, we trained on 2,000 uniformly generated samples. MSE = mean squared error; FO = front occupation; FS = front spread.

4.3 Experiment 2: Activation Function

The purpose of this experiment is study the impact of the three activation functions discussed in Section 3.1.1. We experimented with two solvers, Gurobi Optimization, LLC (2022) and Kuchcinski and Szymanek (2013). The performance of Gurobi is expected to indicate the performance of a state-of-the-art solver, whereas JaCoP should provide insight into how architectural choice can impact performance. While JaCoP is not a commercial solver, unlike Gurobi, it does support modeling of the exponential quadratic term.

Table 6 provides the results. First, we notice Gurobi tends to outperform JaCoP. This shows the underlying solver has a large impact on the performance.

Second, if we consider networks of the same network complexity, sigmoid tends to score best for reducing the mean squared error (MSE), followed by Power2, followed by Taylor1. However, this comes with a tradeoff. Sigmoid introduces an exponential quadratic term, which is not supported by the combination of Gurobi and MiniZinc, forcing the use of the slower JaCoP solver.

Third, Taylor1 is significantly faster than the other two, allowing us to use more complex networks. This shows the potential of using such output activation functions for NN embeddings in EML. The complexity of the network can be increased to an extent where it outperforms sigmoid in terms of accuracy, with no loss in runtime. However, there is a tradeoff. The proposed solutions are not as diverse as those found by the other activation functions. We suspect this to be a cause of a less robust network: Taylor1 scores better on average, but worse in worst-case scenarios.

Lastly, Table 6 shows that using deeper and wider networks improves accuracy, but comes with a tradeoff against increased runtime. The difference can be significant: going from layer width 7 to 10 increases the runtime.

Table 7. Six Nondominating Solutions Found for the Food Wars Experiment.

Input		Output					
		Strength		Mortality Rate		Proactive Aggression	
Collaboration Bonus	Max Age	CP	SIM	CP	SIM	CP	SIM
3.183	271	0.4009	0.4 ± 0.03	0.229	0.21 ± 0.01	0.4965	0.51 ± 0.02
3.184	270	0.4010	0.4 ± 0.02	0.229	0.21 ± 0.01	0.4967	0.48 ± 0.03
3.712	215	0.4030	0.4 ± 0.02	0.237	0.21 ± 0.02	0.5006	0.5 ± 0.03
4.871	158	0.4072	0.44 ± 0.05	0.226	0.24 ± 0.03	0.5096	0.51 ± 0.03
4.955	118	0.4396	0.43 ± 0.02	0.270	0.23 ± 0.08	0.5165	0.51 ± 0.07
4.988	117	0.4401	0.42 ± 0.06	0.268	0.22 ± 0.05	0.5168	0.49 ± 0.06

Note. Initially, 30 instances were generated, each based on a different set of weights. These were solved by constraint programming (CP), of which a set of six nondominating solutions could be constructed. Next, these solutions were sent to the simulation (SIM) to get the actual values. According to the SIM values, these 13 solutions form a nondominating set. Both NNs were trained for 10 min on 5,000 samples. The NN to predict the output variables had two hidden layers of 20 nodes each. The NN to predict the average population had two hidden layers of 15 nodes each. p was set at 100.

4.4 Experiment 3: Food Wars

The purpose of Experiment 3 is to assess the approach on a more complex problem domain. This experiment is based on the multi-agent based Fruit Wars NetLogo model (Rasmussen and Wilensky, 2019) and demonstrates how parameters in nonzero-sum economic environments impact cooperation and violence among agents. In this model, the agents collect fruit in order to survive. They do so by either wandering the map searching for fruit bushes or by attacking other agents to steal their fruit. Agents that survive pass on characteristics that influence their behavior to offspring. Input parameters, such as the bonus gained through collaborating or the maximum age foragers can achieve, are expected to have an impact on traits of the population.

4.4.1 Food Wars—Problem Description. Consider a governing position of some area of fruit foragers. The main objective is to let the foragers behave more peacefully. However, taking governing actions comes with a cost. As such, we are aiming for a proper tradeoff.

One can take action in two dimensions. First, one can choose to be accommodated in healthcare facilities, reflected by the *max-age* parameter. Second, one can give foragers who collaborate a bonus, reflected by the *collaboration_bonus* parameter. As an additional constraint, we want the average population, measured over a certain timespan, to be at least p.

The peacefulness of the population's behavior is indicated by *strength* (how evolution resulted in a population with more strength), *mortality-rate* (how often foragers get killed), and *proactive-aggression* (how often foragers start to attack other foragers). A peaceful population has low values for all of them.

Additionally, it is essential that not all foragers die as a result of our policy. Therefore, we add an extra custom constraint, stating that the number of foragers should at least exceed 100 at the end of the simulation cycle.

Summarized, we are interested in minimizing both the input parameters, *max-age* and *collaboration-bonus*, and minimizing the output parameters *strength*, *mortality-rate*, and *proactive-aggression*, while maintaining a solid population.

4.4.2 Results. Table 7 shows results for this experiment. The solutions with a higher mortality rate are quite possibly a cause of lowering the max age. A lower max age would mean percentage-wise fewer people die of old age, thus the percentage of people that die because of other causes, such as being attacked, increases. The increased strength and proactive aggression are, however, less straightforward to explain. This can very possibly be due to noisy observations, as the simulation outputs tend not to show a consistent pattern.

Although the outputs tend not to differ drastically, the inputs do. This might indicate that the optimal solutions have a similar state, but that they can be achieved in several ways. Differently stated, increasing the collaboration bonus has a similar effect as increasing max age and vice-versa. This should help assist the DM to make a proper decision, where it can take into account how costly it thinks increasing the input parameters would be.

Furthermore, trivial examples where the inputs are minimized toward extreme values are not included. This is an effect of the additional constraint regarding the lower bound for the population. According to our CP model, feeding inputs that strictly dominate the found solutions (i.e., inputs where both max-age and collaboration bonus are lower with respect to at least one of the found solutions), will result in killing too many foragers.

4.5 Discussion

The experiments reported have offered various insights which can be summarized as follows:

- Using low-discrepancy sampling, in our approach through Montgomery sampling, to generate a set of problem instances results in more qualitative and diverse Pareto approximations.
- Deeper and wider networks tend to provide more accuracy at a cost of increased runtime.
- Discarding quadratic terms from output activation functions significantly decreased runtime. Our experiments show that this was paired with a cost of decreased solution diversity.
- The approach can be applied to a diverse range of applications due to its automatic procedure and flexibility in modeling the problem.

5 Conclusion and Future Work

For enterprise SO, this article addressed the crucial question of how to find optimal simulation inputs more effectively. The proposed approach adopts a meta-model in the form of an NN to capture simulation behavior, and embeds the NN automatically into a soft CP model.

We demonstrated the performance sensitivity of the approach by experiments on three problems. In contrast to our previous work, we directly optimize toward a weighted objective instead of the possibilistic approach. We argued in Section 3.3 why this better fits the approach.

The experiments demonstrated how Montgomery sampling reduces the number of simulation calls necessary to obtain the same amount of insight in a stable and deterministic manner. Although for this application the results have shown to be effective, the discrepancy properties—which Sobol and Halton sequences have—are not kept intact. It can be interesting to experiment with an alternative sequence that maintains these properties while adhering to the summation constraint. This is left as future work; we provide a description of an algorithm that could potentially work in Appendix A.

In a highly complex real-world scenario, the number of parameters and observables can be much bigger compared to the experiments performed for this article. A major impact on performance is the solution space—number of input variables and objectives—of the CSP. If the DM is encountering performance issues, we would advise the DM, instead of creating a "master CSP" that addresses all various problems at once, to focus on a subset of input variables and objectives. A good strategy here might be splitting up the CSP and solving the shared CSPs independently.

Furthermore, in this article, we showed activation function Taylor1 performs remarkably well compared to Power2 and Sigmoid in terms of reducing the MSE of the trained network. This shows the potential of using linear-oriented activation functions, when the NN is to be embedded in a CSP. However, these activation functions underperformed in terms of solution diversity. We suspect this to be the cause of network robustness. This issue might be overcome by using a different NN architecture or training strategy. We should note that we only considered a single alternative to an output activation function without a quadratic term. As mentioned in Section 3.1.1, the sigmoid was the only activation function among those experimented with that has a different input for each output and vice versa. It can be interesting to experiment with other activation functions that also have this property and evaluate whether these are more robust. This is left as future work for the EML community.

Building on the approach of this article, it can be interesting to consider constraint acquisition (Bessiere et al., 2017). Here, the CP model is allowed to make simulation calls to obtain more knowledge when it considers it possesses too little. This gives up a form of flexibility, in that solving becomes dependent on the simulation runtime. However, it is expected to be more accurate than our approach as it can consult the simulation model in case of doubt. Constraint acquisition is different from black box optimization, and allows exploitation of the model structure (see Table 2). An interesting follow-up study would be a performance comparison between the proposed methodology, constraint acquisition, and other multi-criteria optimization methods such as evolutionary search.

Acknowledgments

Thanks to the anonymous reviewers. The authors thank Aaron M. Montgomery for his help designing the weight sampling method and Marco Loog for his input on activation functions.

Funding

The authors disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This research was partially supported by TAILOR, a project funded by the EU Horizon 2020 research and innovation programme under grant number 952215.

Declaration of Conflicting Interests

The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

ORCID iDs

Sytze P. E. Andringa https://orcid.org/0000-0003-4061-7104
Neil Yorke-Smith https://orcid.org/0000-0002-1814-3515

Notes

1. We did some minor experimenting to assess performance. We did not notice any noteworthy performance impact when considering equal problem and NN complexity. Hence, in terms of performance, there is no clear winner between the possibilistic and weighted composite model.
2. Algorithm design by Aaron M. Montgomery.

References

Andringa, S. P. E. (2021). *Applying constraint programming to enterprise modelling* [Master's Thesis]. Delft University of Technology.

Andringa, S. P. E., & Yorke-Smith, N. (2021). Flexible enterprise optimization with constraint programming. In *Proceedings of 11th Enterprise Engineering Working Conference (EEWC)* (pp. 58–73). Springer.

Atkinson, C., & Kuhne, T. (2003). Model-driven development: A metamodeling foundation. *IEEE Software*, *20*(5), 36–41. https://doi.org/10.1109/MS.2003.1231149

Audet, C., Bigeon, J., Cartier, D., Le Digabel, S., & Salomon, L. (2021). Performance indicators in multiobjective optimization. *European Journal of Operational Research*, *292*(2), 397–422. https://doi.org/10.1016/j.ejor.2020.11.016

Bartolini, A., Lombardi, M., Milano, M., & Benini, L. (2011). Neuron constraints to model complex real-world problems. In *Proceedings of 17th International Conference on Principles and Practice of Constraint Programming (CP)* (pp. 115–129). Springer.

Bessiere, C., Koriche, F., Lazaar, N., & O'Sullivan, B. (2017). Constraint acquisition. *Artificial Intelligence*, *244*, 315–342. https://doi.org/10.1016/j.artint.2015.08.001

Bock, A., Kudryavtsev, D., & Kubelskiy, M. (2018). Towards more expressive problem structuring: A theoretical conceptualization of 'problem' in the context of enterprise modeling. In *Proceedings of 20th Conference on Business Informatics (CBI'18)* (pp. 30–39). IEEE.

Brachman, R. J., & Levesque, H. J. (2004). The tradeoff between expressiveness and tractability. In *Knowledge representation and reasoning* (pp. 327–348). Morgan Kaufmann.

Buscemi, M. G., & Montanari, U. (2008). A survey of constraint-based programming paradigms. *Computer Science Review*, *2*(3), 137–141. https://doi.org/10.1016/j.cosrev.2008.10.001

Caflisch, R. E. (1998). Monte Carlo and quasi-Monte Carlo methods. *Acta Numerica*, *7*, 1–49. https://doi.org/10.1017/S0962492900002804

Cervellera, C., Gaggero, M., & Macció, D. (2014). Low-discrepancy sampling for approximate dynamic programming with local approximators. *Computers & Operations Research*, *43*, 108–115. https://doi.org/10.1016/j.cor.2013.09.006

Dubey, S. R., Singh, S. K., & Chaudhuri, B. B. (2021). A comprehensive survey and performance analysis of activation functions in deep learning. *CoRR*, abs/2109.14545.

Grossmann, I. E. (2014). Challenges in the application of mathematical programming in the enterprise-wide optimization of process industries. *Theoretical Foundations of Chemical Engineering*, *48*(5), 555–573. https://doi.org/10.1134/S0040579514050182

Gurobi Optimization, LLC (2022). Gurobi optimizer reference manual. https://www.gurobi.com

Halton, J. H. (1960). On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerical Mathematics*, *2*(1), 84–90. https://doi.org/10.1007/BF01386213

IBM (2021). Mathematical programming versus constraint programming, DOcplex v2.22 documentation.

Jimenez-Ramirez, A., Barba, I., Del Valle, C., & Weber, B. (2012). Generating multi-objective optimized configurable business process models. In *Proceedings of 6th International Conference on Research Challenges in Information Science (RCIS)* (pp. 1–2). IEEE.

Jin, R., Chen, W., & Simpson, T. (2001). Comparative studies of metamodelling techniques under multiple modelling criteria. *Structural and Multidisciplinary Optimization*, *23*(1), 1–13. https://doi.org/10.1007/s00158-001-0160-4

Kampik, T., & Najjar, A. (2019). Integrating multi-agent simulations into enterprise application landscapes. In *Proceedings of 2nd International Workshop on Blockchain Technology for Multi-Agent Systems (BTC4MAS), PAAMS 2019 Workshops* (pp. 100–111). Springer.

Kuchcinski, K., & Szymanek, R. (2013). JaCoP – Java constraint programming solver. In *Abstract from CP Solvers: Modeling, Applications, Integration, and Standardization, co-located with the 19th International Conference on Principles and Practice of Constraint Programming (CP)*. https://lup.lub.lu.se/search/ws/files/6233530/4092009.pdf

Laguna, M., & Marklund, J. (2013). Optimizing business process performance. In *Business process modeling, simulation and design* (2nd ed., pp. 439–472). Chapman and Hall.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*(7553), 436–444. https://doi.org/10.1038/nature14539

Lemieux, C. (2009). *Monte Carlo and quasi-Monte Carlo sampling*. Springer.

Lombardi, M., Milano, M., & Bartolini, A. (2017). Empirical decision model learning. *Artificial Intelligence*, *244*, 343–367. https://doi.org/10.1016/j.artint.2016.01.005

Nesterov, Y., & Nemirovskii, A. (1994). *Interior-point polynomial algorithms in convex programming*. Society for Industrial and Applied Mathematics.

Nethercote, N., Stuckey, P., Becket, R., Brand, S., Duck, G., & Tack, G. (2007). MiniZinc: Towards a standard CP modelling language. In *Proceedings of 13th International Conference on Principles and Practice of Constraint Programming (CP)* (pp. 529–543). Springer.

Perron, L., & Furnon, V. (0000). Or-tools, Google. https://developers.google.com/optimization/

Rasmussen, L., & Wilensky, U. (2019). Netlogo fruit wars model. ccl.northwestern.edu/netlogo/models/FruitWars

Rossi, F. (2000). Constraint (logic) programming: A survey on research and applications. In *New trends in constraints* (pp. 40–74). Springer.

Schiex, T. (1992). Possibilistic constraint satisfaction problems or 'how to handle soft constraints?'. In *Proceedings of the 8th Conference on Uncertainty in Artificial Intelligence (UAI)* (pp. 268–275). Morgan Kaufmann.

Schlosser, R., Walther, C., Boissier, M., & Uflacker, M. (2019). Automated repricing and ordering strategies in competitive markets. *AI Communications*, *32*(1), 15–29. https://doi.org/10.3233/AIC-180603

Sobol, I. (1967). On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics*, *7*(4), 86–112. https://doi.org/10.1016/0041-5553(67)90144-9

Tan, J., Braubach, L., Jander, K., Xu, R., & Chen, K. (2020). A novel multi-agent scheduling mechanism for adaptation of production plans in case of supply chain disruptions. *AI Communications*, *33*(1), 1–12. https://doi.org/10.3233/AIC-200646

Teerasoponpong, S., & Sopadang, A. (2021). A simulation-optimization approach for adaptive manufacturing capacity planning in small and medium-sized enterprises. *Expert Systems with Applications*, *168*, 114451. https://doi.org/10.1016/j.eswa.2020.114451

Varma, V., Reklaitis, G., Blau, G., & Pekny, J. (2007). Enterprise-wide modeling & optimization: An overview of emerging research challenges and opportunities. *Computers & Chemical Engineering*, *31*, 692–711. https://doi.org/10.1016/j.compchemeng.2006.11.007

Vernadat, F. (2020). Enterprise modelling: Research review and outlook. *Computers in Industry*, *122*, 103265. https://doi.org/10.1016/j.compind.2020.103265

Wallace, M. (2020). *Building decision support systems*. Springer.

Wiśniewski, P., Kluza, K., Jemioło, P., Ligęza, A., & Suchenia, A. (2021). Business process recomposition as a way to redesign workflows effectively. In *Proceedings of the 16th Conference on Computer Science and Intelligence Systems (FedCSIS)* (pp. 471–474). IEEE.

Yalcin, A., Kilic, H., & Delen, D. (2022). The use of multi-criteria decision-making methods in business analytics: A comprehensive literature review. *Technological Forecasting and Social Change*, *174*, 121193. https://doi.org/10.1016/j.techfore.2021.121193

Zhou, Y.-D., Fang, K.-T., & Ning, J.-H. (2013). Mixture discrepancy for quasi-random point sets. *Journal of Complexity*, *29*(3), 283–301. https://doi.org/10.1016/j.jco.2012.11.006

Appendix A Improved Montgomery Sampling Algorithm[2]

(1) Generate many points from the Halton sequence, that is, $\{\mathbf{h}_j\}, j = 1 \ldots n$, in $\mathbb{R}^{d-1}$; embed them into $\mathbb{R}^d$ by setting their last coordinate to 0.

(2) Let $\{e_i\}$ denote the standard basis for $\mathbb{R}^n$. Construct a $d \times d$ matrix $\mathbf{M}_d$ as follows:
- The rightmost column should be $1/\sqrt{d}$.
- In all other columns:
  * The primary subdiagonal entry in column k should be $k/\sqrt{2s_k}$, where $s_k := \sum_{j=1}^{k} j$ is the kth triangular number.
  * All entries above the primary subdiagonal should be $-1/\sqrt{2s_i}$.
  * All entries below the primary subdiagonal should be 0.

Some examples:

$$d = 3: \qquad \mathbf{M}_3 = \begin{pmatrix} -\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{3}} \\ 0 & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{3}} \end{pmatrix},$$

$$d = 6: \qquad \mathbf{M}_6 = \begin{pmatrix} -\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{12}} & -\frac{1}{\sqrt{20}} & -\frac{1}{\sqrt{30}} & \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{12}} & -\frac{1}{\sqrt{20}} & -\frac{1}{\sqrt{30}} & \frac{1}{\sqrt{6}} \\ 0 & \frac{2}{\sqrt{6}} & -\frac{1}{\sqrt{12}} & -\frac{1}{\sqrt{20}} & -\frac{1}{\sqrt{30}} & \frac{1}{\sqrt{6}} \\ 0 & 0 & \frac{3}{\sqrt{12}} & -\frac{1}{\sqrt{20}} & -\frac{1}{\sqrt{30}} & \frac{1}{\sqrt{6}} \\ 0 & 0 & 0 & \frac{4}{\sqrt{20}} & -\frac{1}{\sqrt{30}} & \frac{1}{\sqrt{6}} \\ 0 & 0 & 0 & 0 & \frac{5}{\sqrt{30}} & \frac{1}{\sqrt{6}} \end{pmatrix}.$$

We can show, for example, by induction, that the columns of $\mathbf{M}_d$ form an orthonormal basis of $\mathbb{R}^d$.

(3) Transform the Halton sequence $\{\mathbf{h}_j\}$ by $T(\mathbf{h}_j) = \sqrt{2}\mathbf{M}_d\mathbf{h}_j + \mathbf{e}_1$, where $\mathbf{e}_1$ is the first vector in the standard basis for $\mathbb{R}^d$ (i.e., has a 1 in the first coordinate and 0 for all others).
- The logic behind this is that the rotation matrix $\mathbf{M}_d$ maps the subspace $\mathbb{R}^{d-1} \times \{\mathbf{0}\}$ (i.e., the subspace containing the generated Halton set) onto the space orthogonal to $\mathbf{1}$, that is, the subspace for which the sum of the coordinates is 0. This mapping does not distort distances.
- The $\sqrt{2}$ term is to ensure that the mapping $[0, 1]^{d-1} \times \{\mathbf{0}\} \to S$ is surjective, since $T(\mathbf{e}_1) = (0, 1, 0, \dots)$.

(4) Many of the points transformed will not have all coordinates in $[0, 1]$ as desired; discard those that do not. Roughly $1/d!$ of the terms in $\{\mathbf{j}_j\}$ will correctly map to S. So if we would like n points on S, that would require about $n \cdot d!$ points generated on $[0, 1]^{d-1}$.

Appendix B NN Embedding in MiniZinc, Supply Chain Experiment

```
var float: lost_ratio_sales_ret;
var float: lost_ratio_sales_dis;
var float: lost_ratio_sales_fac;
var float: n_0_0Supply_Chain_Supply_Chain_halton_pow2;
constraint (n_0_0Supply_Chain_Supply_Chain_halton_pow2 = max((-0.18071165680885315 * Fact + -0.1765439808368683 * Distr1 + 0.08369018137454987 * Distr2 +
      0.25116947293281555 * Clients_N + -0.1845000982284546 * Demand_W + 0.21171733736991882 * HC + 0.2007092982530594 * Lt0 + 0.19539859890937805 * Lt1 +
      -0.11898324638605118 * SS_ + -0.2221519500017166 * DS_D + -0.20919005572795868 * K + 0.06167382001876831 * MA + 0.1683061122894287 * v1_DailyPurchase +
      0.12330305576324463 * v2_PeriodicallyPurchase + 0.19629697501659393 * v3_Random + 0.14650665223598 48 * v1_s_Q + -0.21137376129627228 * v2_s_S +
      0.17657363414764404 * v3_R_S + 0.29423147439956665 * v4_Random + -0.09469400346279144), 0));
...
var float: n_0_11Supply_Chain_Supply_Chain_halton_pow2;
constraint (n_0_11Supply_Chain_Supply_Chain_halton_pow2 = max((-0.09817271679639816 * Fact + -0.1110820546746254 * Distr1 + -0.1972866952419281 * Distr2 +
      -0.153198286890983 58 * Clients_N + 0.16212717222982 87 * Demand_W + 0.13568077981472015 * HC + -0.10288749635219574 * Lt0 + -0.10385935008525848 * Lt1 +
      -0.08175746351480484 * SS_ + -0.14531876146793365 * DS_D + 0.05007623881101608 * K + 0.18467850983142853 * MA + 0.03140450268983841 * v1_DailyPurchase +
      -0.051365479826927185 * v2_PeriodicallyPurchase + -0.2115851491689682 * v3_Random + 0.33020681142807007 * v1_s_Q + 0.20140214264392853 * v2_s_S +
      0.006390944588929415 * v3_R_S + -0.19572050869464874 * v4_Random + 0.15225890278816223), 0));
var float: n_1_0Supply_Chain_Supply_Chain_halton_pow2;
constraint (n_1_0Supply_Chain_Supply_Chain_halton_pow2 = max((-0.02960401587188244 * n_0_0Supply_Chain_Supply_Chain_halton_pow2 + -0.10777605324983597 *
      n_0_1Supply_Chain_Supply_Chain_halton_pow2 + 0.05967755988240242 * n_0_2Supply_Chain_Supply_Chain_halton_pow2 + -0.20008115470409393 *
      n_0_3Supply_Chain_Supply_Chain_halton_pow2 + -0.04289616644382477 * n_0_4Supply_Chain_Supply_Chain_halton_pow2 + 0.22794991731643677 *
      n_0_5Supply_Chain_Supply_Chain_halton_pow2 + 0.21227192878723145 * n_0_6Supply_Chain_Supply_Chain_halton_pow2 + 0.2164132297039032 *
      n_0_7Supply_Chain_Supply_Chain_halton_pow2 + -0.009562408551573753 * n_0_8Supply_Chain_Supply_Chain_halton_pow2 + -0.007423806004226208 *
      n_0_9Supply_Chain_Supply_Chain_halton_pow2 + 0.028055837377905846 * n_0_10Supply_Chain_Supply_Chain_halton_pow2 + 0.3192550241947174 *
      n_0_11Supply_Chain_Supply_Chain_halton_pow2 + 0.20619946718215942), 0));
...
var float: n_1_11Supply_Chain_Supply_Chain_halton_pow2;
constraint (n_1_11Supply_Chain_Supply_Chain_halton_pow2 = max((0.08487904071807861 * n_0_0Supply_Chain_Supply_Chain_halton_pow2 + 0.0187103021889925 *
      n_0_1Supply_Chain_Supply_Chain_halton_pow2 + 0.024226030334830284 * n_0_2Supply_Chain_Supply_Chain_halton_pow2 + -0.0021953957621008158 *
      n_0_3Supply_Chain_Supply_Chain_halton_pow2 + 0.1034083217382431 * n_0_4Supply_Chain_Supply_Chain_halton_pow2 + -0.30843567848205566 *
      n_0_5Supply_Chain_Supply_Chain_halton_pow2 + -0.08998899161815643 * n_0_6Supply_Chain_Supply_Chain_halton_pow2 + -0.1375928372144699 *
      n_0_7Supply_Chain_Supply_Chain_halton_pow2 + -0.07159772515296936 * n_0_8Supply_Chain_Supply_Chain_halton_pow2 + -0.29670989513397217 *
      n_0_9Supply_Chain_Supply_Chain_halton_pow2 + 0.25440090894699097 * n_0_10Supply_Chain_Supply_Chain_halton_pow2 + 0.07540704309940338 *
      n_0_11Supply_Chain_Supply_Chain_halton_pow2 + -0.014821779914200306), 0));


var float: n_2_0Supply_Chain_Supply_Chain_halton_pow2;
constraint (n_2_0Supply_Chain_Supply_Chain_halton_pow2 = (0.18654236197471 62 * n_1_0Supply_Chain_Supply_Chain_halton_pow2 + 0.25312069058418274 *
      n_1_1Supply_Chain_Supply_Chain_halton_pow2 + -0.30259981751441956 * n_1_2Supply_Chain_Supply_Chain_halton_pow2 + -0.5652911067008972 *
      n_1_3Supply_Chain_Supply_Chain_halton_pow2 + 0.00649289321154356 * n_1_4Supply_Chain_Supply_Chain_halton_pow2 + 0.04832885414361954 *
      n_1_5Supply_Chain_Supply_Chain_halton_pow2 + -0.2324041724205017 * n_1_6Supply_Chain_Supply_Chain_halton_pow2 + 0.07574844360351562 *
      n_1_7Supply_Chain_Supply_Chain_halton_pow2 + -0.41326600313186646 * n_1_8Supply_Chain_Supply_Chain_halton_pow2 + 0.2799031436443329 *
      n_1_9Supply_Chain_Supply_Chain_halton_pow2 + -0.1949828714132309 * n_1_10Supply_Chain_Supply_Chain_halton_pow2 + -0.022231847047805786 *
      n_1_11Supply_Chain_Supply_Chain_halton_pow2 + 0.038756951689720154));
...
var float: n_2_2Supply_Chain_Supply_Chain_halton_pow2;
constraint (n_2_2Supply_Chain_Supply_Chain_halton_pow2 = (-0.010642621666193008 * n_1_0Supply_Chain_Supply_Chain_halton_pow2 + 0.09052400290966034 *
      n_1_1Supply_Chain_Supply_Chain_halton_pow2 + -0.1101236492395401 * n_1_2Supply_Chain_Supply_Chain_halton_pow2 + 0.01919526420533657 *
      n_1_3Supply_Chain_Supply_Chain_halton_pow2 + -0.09344010055065155 * n_1_4Supply_Chain_Supply_Chain_halton_pow2 + 0.29708153009414673 *
      n_1_5Supply_Chain_Supply_Chain_halton_pow2 + 0.032747652381658554 * n_1_6Supply_Chain_Supply_Chain_halton_pow2 + 0.1762314736843109 *
      n_1_7Supply_Chain_Supply_Chain_halton_pow2 + 0.07827277481555939 * n_1_8Supply_Chain_Supply_Chain_halton_pow2 + 0.04886416718363762 *
      n_1_9Supply_Chain_Supply_Chain_halton_pow2 + -0.009062468074262142 * n_1_10Supply_Chain_Supply_Chain_halton_pow2 + 0.08940587937831879 *
      n_1_11Supply_Chain_Supply_Chain_halton_pow2 + -0.11300095915794373));
constraint (lost_ratio_sales_ret * (1 + n_2_0Supply_Chain_Supply_Chain_halton_pow2 * n_2_0Supply_Chain_Supply_Chain_halton_pow2) = 1);
constraint (lost_ratio_sales_dis * (1 + n_2_1Supply_Chain_Supply_Chain_halton_pow2 * n_2_1Supply_Chain_Supply_Chain_halton_pow2) = 1);
constraint (lost_ratio_sales_fac * (1 + n_2_2Supply_Chain_Supply_Chain_halton_pow2 * n_2_2Supply_Chain_Supply_Chain_halton_pow2) = 1);

```